SOEN 343 - Phase 2
System Architecture, Use Case and Sequence Diagram

SUMMS - Smart Urban Mobility Management System

Team name: TABAC

Team Leader: Daniel Ganchev - 40315755
Developer 1: Adam Ousmer - 40246695
Architect 1: Dylan Bourret - 40287207
Developer 2: Yanis Djeridi - 40227313
Architect 2: Emre Emuler - 40212481
Tester: Abed-Elmouhsen Cherkawi - 40323359

Thursday February 26th 2026

Introduction 3
1. System Architecture 3
1.1 Selected Architecture: Layered MVC Architecture (Django - MVT pattern) 3
1.1.1 Alignment with Phase 1 Requirements 4
1.2 Architectural Structure 4
1.2.1 Presentation Layer (Django Templates) 5
1.2.2 Controller Layer (Django Views) 6
1.2.3 Business Logic Layer (Core Intelligence) 6
Vehicle Rental Management 6
Analytics 7
Parking 7
Public Transit Module 7
Gamification + Sustainability 8
1.2.4 Data Layer 8
1.2.5 Component Interaction 9
1.3 External System Integration 9
1.4 Architectural Qualities Prioritized 9
1.4.1 Maintainability 9
1.4.2 Scalability 9
1.4.3 Performance 10
1.4.4 Security 10
1.4.5 Extensibility 10
1.5 Alternative Architecture Considered: Microservices 10
1.5.1 Strengths 10
1.5.2 Trade-offs 11
1.6 Final Justification 11
2 Use Cases 12
2.1 Use Case Diagram 12
2.2 Use Case Scenarios 13
3 Sequence Diagram 20

Introduction

This phase targets:
1. Understanding the system architecture.
2. Understanding the use case diagram.
3. Understanding the sequence diagram.

Assigned Tasks:

Daniel	System Architecture
Adam	DB, backend
Dylan	Use case diagram, Frontend Wireframe
Yanis	DB, backend
Emre	Sequence diagram
Abed	Use case Scenarios

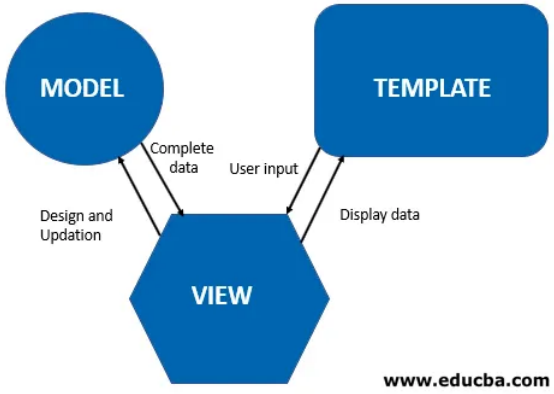
1. System Architecture

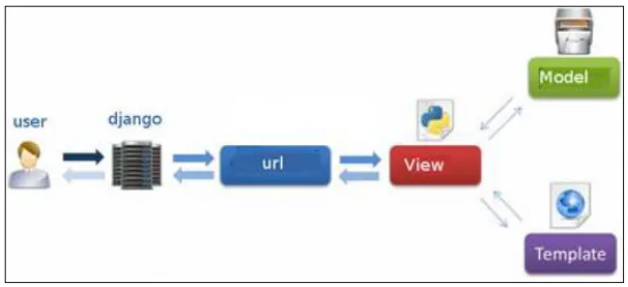
1.1 Selected Architecture: Layered MVC Architecture (Django - MVT pattern)

SUMMS is implemented using a layered architecture following Django's Model-View-Template (MVT) pattern.

Note: in Django, different naming conventions:

- Model is the same (handles data and business logic).
- View (MVT) acts as the controller (processes requests, applies logic, selects templates)
- Template handles presentation





Here's a high-level overview of the architecture:

- **Core Focus**: Vehicle Rental Lifecycle + Advanced Analytics
- **Secondary Module:** Parking Management
- **Additional Feature**: Public Transit Integration
- **Originality**: Gamified reliability system + sustainability analytics

Because Django follows the MVT structure, it allows clear separation between presentation, business logic, and data while remaining suitable for rapid development and future scalability.

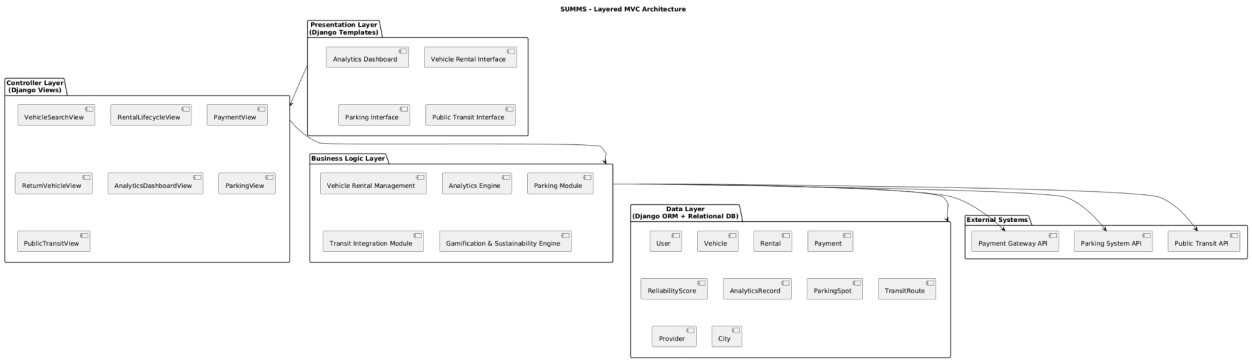
This architectural choice ensures that the **vehicle rental and analytics engines are treated as core subsystems**, not just features.

1.1.1 Alignment with Phase 1 Requirements

The selected layered MVC architecture directly supports Phase 1 requirements:

- Integrated mobility services → Centralized business logic coordinates rentals, parking, and transit access consistently.
- Rental lifecycle control → Business layer enforces reservation, payment, unlock, and return rules.
- Analytics requirements → Dedicated analytics module computes rentals per city, mobility type, and gateway statistics.
- Admin monitoring → Separate dashboards supported by analytics layer.
- Personalization and sustainability → Business layer handles sustainability score and user-specific data without affecting other layers.

1.2 Architectural Structure



The system is organized in <u>four</u> logical layers.

1.2.1 Presentation Layer (Django Templates)

Main Interfaces:

Vehicle Rental Interface

- Vehicle search and filtering
- Reservation and payment confirmation
- Vehicle return
- Rental history and reliability score visualization

Analytics Dashboard

- Rentals (per city, vehicle type)
- Revenue analytics
- Demand forecasting
- CO_2 impact visualization
- gateway service access statistics

Parking Interface

- View parking availability
- Reserve parking spot

Public Transit Interface

- View transit routes
- Combine transit with rental trips

This layer strictly handles visualization and user interaction.

1.2.2 Controller Layer (Django Views)

The controller layer processes user requests and connects them to business logic.

Core Controllers

- VehicleSearchView
- RentalLifecycleView
- PaymentView
- ReturnVehicleView
- AnalyticsDashboardView

Controllers validate input, enforce permissions, invoke service logic, and return HTML or JSON responses.

Secondary Controllers

- ParkingView
- PublicTransitView

Controllers:

- Validate input
- Enforce permissions
- Call service logic
- Return HTML or JSON responses

1.2.3 Business Logic Layer (Core Intelligence)

This layer represents the brain of SUMMS and deals with component communication.

Implemented with:

- Django Models
- Python service modules
- Analytics computation modules

Vehicle Rental Management

Handles the full rental lifecycle:

- Vehicle availability validation

- Reservation creation
- Conflict prevention
- Payment processing
- Vehicle return processing
- Rental history tracking
- Reliability score updates

Business rules enforced here:

- A vehicle cannot be double-booked
- Payments must succeed before activation
- Late returns affect reliability score

Analytics

This component is for city administrators and providers.

Includes:

- Rental volume analytics
- Revenue analytics
- Rentals per city
- Rentals per vehicle type
- gateway analytics (service usage frequency)
- Demand pattern tracking
- Sustainability analytics (CO_2 impact per rental)
- Behavioral analytics for incentive scoring

This engine processes historical rental data stored in the database and generates aggregated metrics.

The analytics module is modular to allow future predictive extensions.

Parking

Handles:

- Parking spot availability
- Reservation validation
- Simple occupancy tracking

Parking remains functionally correct but architecturally lighter.

Public Transit

Handles:

- Transit schedule retrieval
- Route viewing
- Integration into multimodal suggestions

Transit is abstracted as an external integration service.

Gamification + Sustainability

SUMMS includes innovative features:

- Gamified reliability score system
- Incentive engine rewarding responsible usage
- Trip-level CO_2 impact calculation
- Behavioral redistribution incentives

This module interacts with the rental lifecycle and analytics.

1.2.4 Data Layer

Implemented using Django ORM with a relational database.

Main Entities (High Priority)

- User
- Vehicle
- Rental
- Payment
- ReliabilityScore
- AnalyticsRecord

Secondary Entities

- ParkingSpot
- TransitRoute
- Provider
- City

Responsibilities:

- Maintain referential integrity
- Enforce constraints
- Store historical rental data
- Support analytics queries

Database indexing is applied on:

- Vehicle availability
- Rental timestamps
- City
- Vehicle type

This supports analytics performance.

1.2.5 Component Interaction

Controllers invoke business service modules, which enforce system rules and interact with the Django ORM-based data layer.

External systems (payment, parking, and transit APIs) are accessed through REST-based service adapters, ensuring loose coupling between SUMMS and third-party services.

This layered interaction guarantees separation of concerns and controlled data flow across the system.

1.3 External System Integration

SUMMS integrates with:

- Payment gateways
- Parking systems
- Public transit APIs

All communication is REST-based using JSON.

Vehicle rental logic remains internally managed.

1.4 Architectural Qualities Prioritized

1.4.1 Maintainability

Clear separation between rental logic and analytics ensures:

- Analytics updates do not break rental lifecycle
- Parking and transit remain isolated modules

1.4.2 Scalability

The architecture allows:

- Scaling analytics independently
- Database optimization for heavy rental queries
- Future extraction of analytics into separate service if needed

1.4.3 Performance

Optimizations focus primarily on:

- Rental lifecycle responsiveness
- Fast availability queries
- Efficient analytics aggregation
- Indexed queries for rental statistics

1.4.4 Security

- Django authentication
- Role-based permissions (Commuter, Provider, Admin)
- Secure payment handling
- Protection against double-booking attacks

1.4.5 Extensibility

The modular structure allows:

- Adding new vehicle types
- Expanding analytics metrics
- Adding new incentive policies
- Expanding public transit integrations

1.5 Alternative Architecture Considered: Microservices

Microservices were considered for separating:

- Rental service
- Analytics service
- Parking service
- Transit integration

1.5.1 Strengths

- Independent scaling

- Service isolation

1.5.2 Trade-offs

- High infrastructure complexity
- Requires containerization and orchestration

A modular Django monolith provides better maintainability and clarity.

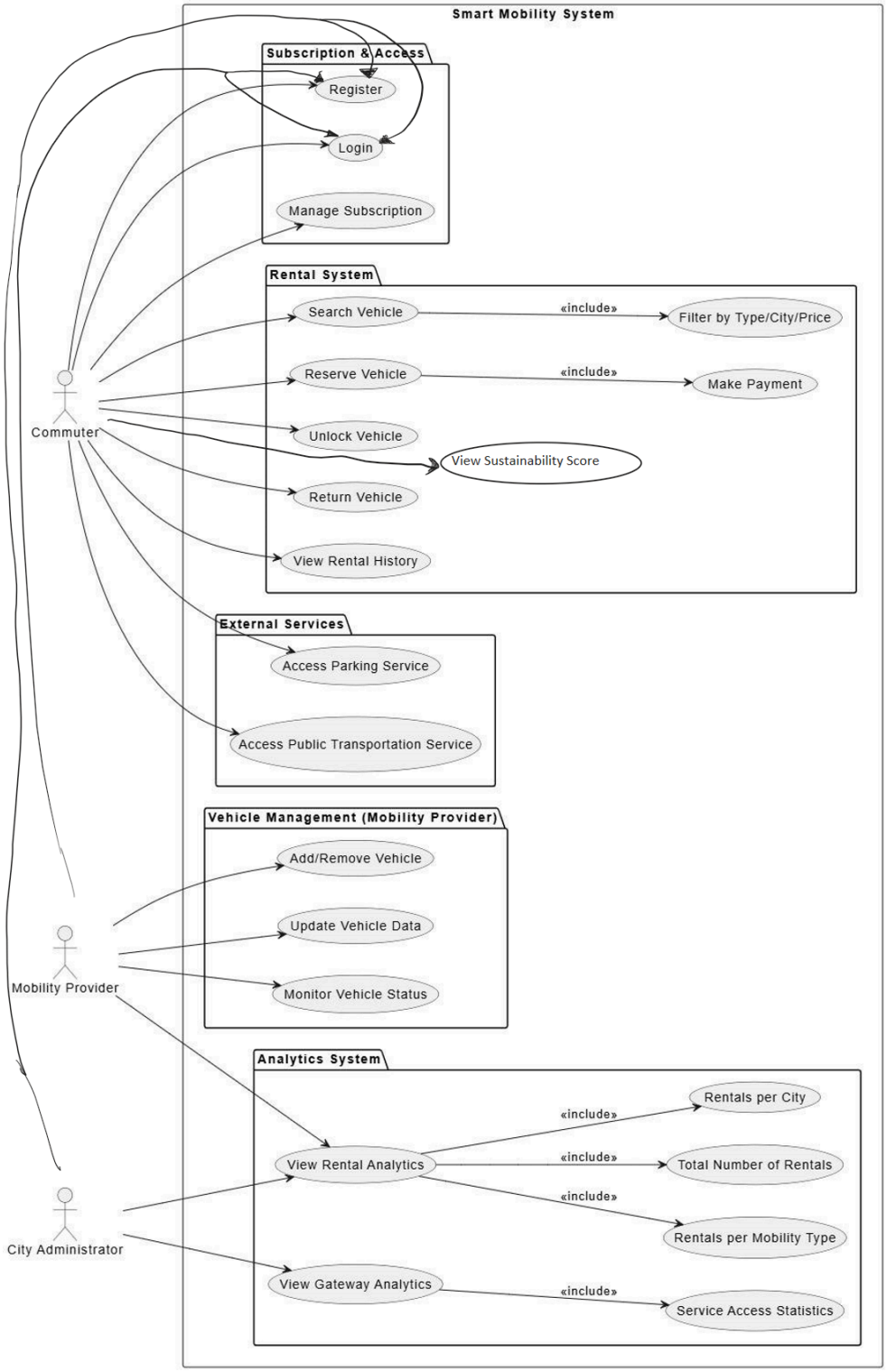
1.6 Final Justification

The selected Layered MVC Architecture:

- Prioritizes Vehicle Rental and Analytics
- Supports Parking and Public Transit as secondary modules
- Enables implementation of originality features (gamification, environmental friendly)
- Simple and allows scalability for future features

2 Use Cases

2.1 Use Case Diagram



2.2 Use Case Scenarios

For each use case described in the above section, provide a use case scenario describing the behavior of the system. Use the below template for your scenarios:

ID:	UC-01
Title:	Register User Account
Description:	Allows a new user (commuter, provider, or administrator) to create an account in SUMMS.
Primary Actor:	Commuter, mobility provider, and city admin.
Preconditions:	User is not authenticated.
Postconditions:	User account is created and stored in the database.
Inputs:	Name, email, password, role selection.
Outputs:	Confirmation message, authenticated session.
Main success Scenario:	1. User selects "Register". 2. System displays registration form. 3. User enters required information. 4. System validates input. 5. The system assigns role-based permissions depending on selected user type. 6. System creates user record. 7. User is redirected to dashboard.

ID:	UC-02
Title:	Search Vehicles
Description:	Allows a commuter to search for available rental vehicles based on selected filters.
Primary Actor:	Commuter
Preconditions:	User is logged in.
Postconditions:	Filtered list of vehicles is displayed.
Inputs:	City, vehicle type, price range, date/time.

Outputs:	List of available vehicles.
Main success Scenario:	1. User accesses search page. 2. User enters filter criteria. 3. System queries database. 4. System retrieves matching vehicles. 5. System displays results.

ID:	UC-03
Title:	Rental Lifecycle
Description:	Represents the full rental process from reservation to vehicle return.
Primary Actor:	Commuter
Preconditions:	User is logged in. Vehicle availability is verified during execution.
Postconditions:	Rental is completed, payment processed, rental record stored, and sustainability metrics (CO_2 savings and reward points) are updated.
Inputs:	Vehicle selection, rental duration, payment details.
Outputs:	Reservation confirmation, payment receipt, rental history update.
Main success Scenario:	1. User selects vehicle. 2. User confirms reservation details. 3. System validates availability. 4. System creates reservation record. 5. User enters payment information. 6. Payment gateway authorizes payment. 7. System confirms reservation. 8. User starts rental. 9. User ends rental. 10. System calculates final charges. 11. System updates rental status to completed. 12. System calculates CO_2 emissions saved for the trip. 13. System updates user sustainability score and reward points.

ID:	UC-04
Title:	Access Parking Service
Description:	Allows users to view and reserve parking spaces through integrated parking system.
Primary Actor:	Commuter

Preconditions:	User is logged in.
Postconditions:	Parking reservation confirmed.
Inputs:	Location, date/time.
Outputs:	Parking availability list and confirmation.
Main success Scenario:	1. User selects parking option. 2. System queries external parking API. 3. System displays available spots. 4. User selects spot. 5. Reservation confirmation is returned.

ID:	UC-05
Title:	View Public Transit Information
Description:	Allows users to retrieve public transit routes and schedules.
Primary Actor:	Commuter
Preconditions:	User is logged in.
Postconditions:	Transit data displayed.
Inputs:	Origin, destination, date/time.
Outputs:	Transit routes and schedules.
Main success Scenario:	1. User enters route details. 2. System calls transit API. 3. Transit data is retrieved. 4. System displays route options.

ID:	UC-06
Title:	Manage Vehicles
Description:	Allows a mobility provider to add, update, or remove vehicles from the system and manage fleet data.
Primary Actor:	Mobility Provider
Preconditions:	Provider is authenticated and has provider-level access rights.
Postconditions:	Vehicle records are created, updated, or removed in the system database.

Inputs:	Vehicle type, license plate, city, price per hour, availability status.
Outputs:	Confirmation message and updated fleet list.
Main Success Scenario:	1. Provider logs into dashboard. 2. Provider selects "Add Vehicle". 3. System displays vehicle input form. 4. Provider enters vehicle details. 5. System validates input data. 6. System stores vehicle in database. 7. Updated fleet list is displayed. Alternative Flows: ● Update Vehicle ● Delete Vehicle

ID:	UC-07
Title:	View Rental Analytics
Description:	Allows a city administrator to view statistics on a specific rental service.
Primary Actor:	City Administrator
Preconditions:	Administrator is authenticated.
Postconditions:	Rental statistics are displayed in dashboard format.
Inputs:	Date range, city filter, mobility type filter.
Outputs:	Metrics: ● Total number of rentals ● Rentals per city ● Rentals per mobility type ● Peak rental times
Main success Scenario:	1. Administrator selects rental analytics module. 2. Administrator chooses filter criteria. 3. System retrieves rental data from database. 4. System calculates aggregated metrics. 5. Dashboard charts and summaries are displayed.

ID:	UC-08
Title:	View Gateway Analytics
Description:	Allows the city administrator to monitor usage statistics of each service in

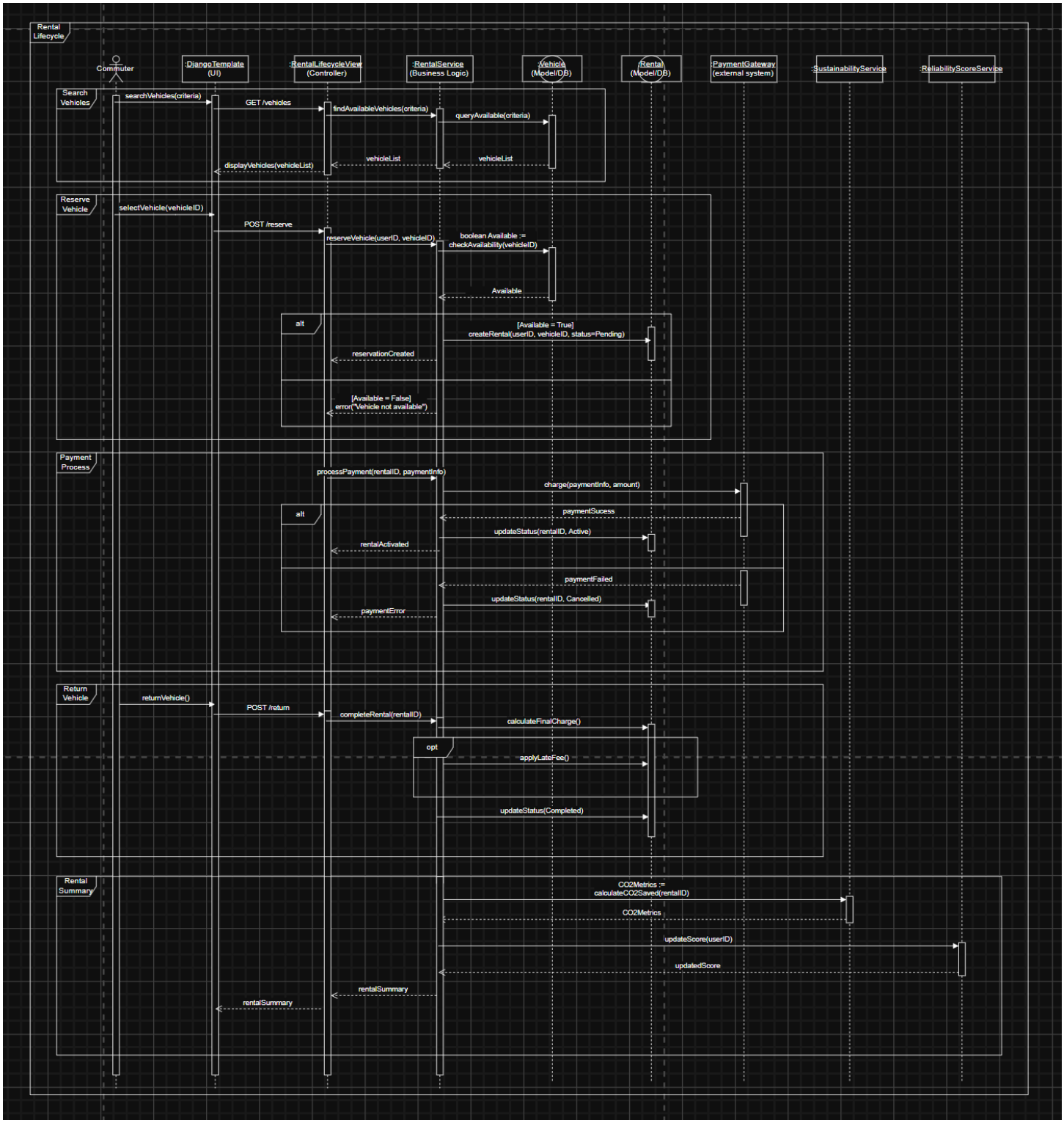
	SUMMS.
Primary Actor:	City Administrator
Preconditions:	Administrator is authenticated.
Postconditions:	Service usage statistics are displayed.
Inputs:	Date range.
Outputs:	Metrics: ● Number of rental service accesses ● Number of parking service accesses ● Number of transit API calls ● System usage frequency
Main success Scenario:	1. Administrator accesses gateway analytics module. 2. System retrieves system access logs. 3. System aggregates API usage data. 4. System displays service access statistics and charts.

ID:	UC-09
Title:	View Sustainability Score
Description:	Allows commuters to view their sustainability performance, including CO_2 savings and earned reward points.
Primary Actor:	Commuter
Preconditions:	User is authenticated and has completed at least one rental.
Postconditions:	User sustainability metrics and rewards are displayed.
Inputs:	User ID, date range (optional).
Outputs:	● Total CO_2 emissions saved ● Green mobility score ● Reward points ● Badge level
Main success Scenario:	1. User accesses sustainability dashboard. 2. System retrieves user's rental history. 3. System calculates CO_2 savings. 4. System computes reward points. 5. System displays sustainability summary and achievements.

ID:	UC-10
Title:	Login
Description:	Allows registered users (Commuter, Mobility Provider, or City Administrator) to authenticate and access the system.
Primary Actor:	Commuter, Mobility Provider, City Administrator
Preconditions:	User account exists in the system and user is not already authenticated.
Postconditions:	User is authenticated and user is redirected to their role-specific dashboard.
Inputs:	Email/User ID, password.
Outputs:	Authentication confirmation or Error message (if credentials are invalid)
Main success Scenario:	1. User selects "Login". 2. System displays login form. 3. User enters credentials. 4. System validates credentials against stored data. 5. System authenticates user. 6. System determines user role. 7. User is redirected to their respective dashboard. Alternative Flow - Invalid Credentials: 4a. Credentials are incorrect. 4b. System displays error message. 4c. User is prompted to retry login.

ID:	UC-11
Title:	Manage Subscription
Description:	Allows users to modify or cancel their subscription plan.
Primary Actor:	Commuter, Mobility Provider, City Administrator
Preconditions:	User is authenticated and user has an active account.
Postconditions:	Subscription plan is updated or cancelled, and changes are stored in the system.
Inputs:	Selected subscription plan and payment details (if upgrading)
Outputs:	Confirmation message and updated subscription status
Main success Scenario:	1. User accesses subscription settings. 2. System displays current subscription details.

	3. User selects new plan or cancellation option. 4. System validates selection. 5. If payment required, system processes payment. 6. System updates subscription record. 7. Confirmation is displayed. Alternative Flow - Payment Failure: 5a. Payment fails. 5b. System displays error message. 5c. Subscription remains unchanged.

ID:	UC-12
Title:	View Rental History
Description:	Allows commuters to view their past and current rentals.
Primary Actor:	Commuter
Preconditions:	User is authenticated and user has at least one rental record.
Postconditions:	Rental history is displayed.
Inputs:	Date range (optional)
Outputs:	List of past rentals and rental details (vehicle type, duration, cost, status)
Main success Scenario:	1. User accesses rental history section. 2. System retrieves rental records from database. 3. System filters records if criteria provided. 4. System displays rental history list.

3 Sequence Diagram